

Élicitation incrémentale des préférences appliquée aux parfums

Raphaël Yzermans Mathieu Laurent Victor Soupramanien
Victor Pezennecc–Deutsch

18 mai 2026

Remerciements

Avant de commencer ce rapport, nous souhaitons exprimer notre reconnaissance à ceux qui ont contribué à la réussite de ce projet. Nous adressons tout particulièrement nos remerciements à Mohamed Ouaguenouni, notre maître de projet, pour sa disponibilité, son écoute et ses conseils tout au long du semestre. Son encadrement nous a permis d'aborder des problématiques complexes avec rigueur, tout en gardant une grande autonomie. Son expertise et sa bienveillance ont grandement facilité notre progression et ont été précieuses pour structurer notre démarche.

Table des matières

Introduction	4
1 Développement de l'interface Web	5
1.1 Technologies utilisées	5
1.2 Justification des choix techniques : Flask et React	5
1.3 Architecture générale	6
1.4 Mise en page et système utilisateur	6
1.5 Fonctionnalités principales	6
1.5.1 Architecture de la partie Backend – Gestion de la base de données	7
1.6 Gestion de l'état utilisateur	7
1.7 Nettoyage et structuration de la base de données	8
1.7.1 Indexation des entités textuelles	8
1.7.2 Préparation de la base de données	8
2 Élicitation des préférences et modélisation IA	10
2.1 Modélisation géométrique des préférences	10
2.2 Optimisation par fonction barrière logarithmique	12
2.3 Estimation locale : Ellipse de confiance	13
2.4 Détermination de la meilleure question à poser	14
2.4.1 Méthode de recherche exhaustive	15
2.4.2 Méthode du critère D-optimality	16
2.5 Critère d'arrêt basé sur le Min-Max Regret	16
3 Bilan du développement : réussites et difficultés	18
3.1 Organisation initiale et réajustements	18
3.2 Difficultés rencontrées	18
3.2.1 Exploration de plusieurs approches non concluantes	18
3.2.2 Lenteur générale de la modélisation au départ	19
3.3 Réussites techniques et méthodologiques	19
3.3.1 Méthode de Newton pour la fonction barrière	19
3.3.2 Recherche exhaustive des meilleures questions	19

TABLE DES MATIÈRES

3.3.3	Implémentation du min-max regret	19
3.3.4	Alternative au regret	20
3.4	Leçons retenues	21
Conclusion		22

Introduction

Dans un monde où le nombre de parfums ne cesse de croître, aider les utilisateurs à trouver un nouveau parfum est souvent une tâche ardue, en particulier dans le domaine de la parfumerie, qui fait intervenir un grand nombre d'ingrédients. Une fois mélangés, ces ingrédients produisent une odeur différente de celle de chaque composant pris individuellement. Ce projet a pour objectif de concevoir un système interactif d'aide à la décision, capable d'identifier progressivement les préférences olfactives d'un utilisateur afin de lui recommander un parfum. Pour cela, nous nous appuyons sur une base de données regroupant environ 2000 parfums, décrits par leurs caractéristiques olfactives. Notre approche repose sur un processus d'élicitation incrémentale des préférences. Concrètement, l'utilisateur est invité à interagir avec une interface web que nous avons développée, en comparant des parfums ou en attribuant des notes. Chaque réponse permet d'affiner la modélisation de ses préférences et de proposer le parfum le plus adapté à ses goûts. L'objectif final est double : offrir une expérience utilisateur fluide et intuitive, tout en garantissant une recommandation pertinente. Ce rapport présente l'ensemble des choix techniques, méthodologiques et algorithmiques qui ont permis de construire ce système, depuis le développement de l'interface jusqu'à l'implémentation des algorithmes d'élicitation.

Chapitre 1

Développement de l'interface Web

1.1 Technologies utilisées

Pour le développement de notre interface, nous avons opté pour un backend en Python avec Flask et un frontend en React.js. L'application est hébergée sur la plateforme Render, un service cloud gratuit et facile à utiliser, qui permet de déployer rapidement une application sans configuration serveur complexe.

1.2 Justification des choix techniques : Flask et React

Nous avons choisi Flask car c'est un micro-framework léger et très simple à prendre en main. Il est particulièrement adapté à des projets comme le nôtre, où nous devons implémenter un modèle de prédiction codé en Python. De plus, Flask est rapide, flexible et facile à maîtriser, ce qui était parfait pour nous, qui n'avions jamais développé de site web auparavant. Grâce à sa structure claire, il nous a permis de créer une API qui communique facilement avec notre interface React.

React est une bibliothèque JavaScript très populaire pour la création d'interfaces utilisateur. Ce que nous avons particulièrement apprécié, c'est sa structure basée sur des composants réutilisables : cela nous a permis de construire rapidement des prototypes, puis de les faire évoluer sans repartir de zéro. React est aussi très bien documenté et permet de concevoir des interfaces agréables rapidement et facilement. Il est parfaitement compatible avec Vite, que nous avons utilisé pour améliorer notre environnement de développement.

React nous a également permis de gérer facilement les routes entre les différentes pages de notre projet. Pour cela, nous avons utilisé une bibliothèque spécifique à React appelée `react-router-dom`, qui nous a permis de structurer les différentes pages de notre application (identification de l'utilisateur, notation, comparaison, résultats) grâce à un système de routage clair et efficace.

L'association Flask/React est idéale pour une architecture frontend/backend bien séparée. Cette séparation nous a permis de travailler indépendamment sur l'interface utilisateur et sur l'élicitation et la modélisation des préférences. C'est une approche très pratique, surtout pour le prototypage et le travail en équipe. Enfin, ce couple technologique est largement utilisé, ce qui nous a permis de trouver facilement des ressources et de l'aide pendant le développement.

Comme nous l'avons dit précédemment, nous avons aussi utilisé Vite, un outil de développement moderne qui permet de lancer rapidement une application React. Contrairement à Webpack, il se distingue par sa vitesse de démarrage et son système de rechargement très efficace, ce qui est très utile pour un projet contenant beaucoup de pages comme le nôtre, car Vite permet de les charger plus rapidement.

1.3 Architecture générale

Comme nous l'avons dit précédemment, nous avons adopté une architecture de type "frontend-backend séparé". Ce modèle nous a permis de bien organiser notre code et de préparer le projet pour de futures évolutions.

1.4 Mise en page et système utilisateur

L'application web a été conçue pour être intuitive et agréable à utiliser. Nous avons adopté une mise en page claire et structurée.

Nous avons mis en place un système de gestion des utilisateurs simple mais fonctionnel. L'utilisateur peut se connecter via une page de *login*. Une fois connecté, il accède à différentes sections de l'application : notation des parfums, comparaison.

Chaque page a été pensée comme un composant indépendant dans React, facilement accessible via `react-router-dom`. Cela inclut :

- une page de connexion/inscription,
- une page de notation des parfums,
- une page de comparaison binaire (préférence),
- une page de résultats personnalisés.

Ce découpage clair nous a permis de structurer l'interface de manière modulaire et de faciliter les tests.

1.5 Fonctionnalités principales

L'utilisateur peut rechercher un parfum qu'il connaît et lui attribuer une note de 1 à 5 étoiles. L'interface permet de choisir entre deux parfums présentés en parallèle, et l'utilisateur choisit celui qu'il préfère (préférence de type "j'aime / je n'aime pas").

1.5.1 Architecture de la partie Backend – Gestion de la base de données

Dans notre projet, le backend est structuré de manière claire et modulaire, ce qui facilite la lecture, la maintenance et l'évolution du code. Cette structure repose principalement sur quatre dossiers : **models**, **controllers**, **routes** et **config**. Chacun de ces dossiers joue un rôle précis dans la gestion de la base de données.

Le dossier **models** contient les fichiers qui interagissent directement avec la base de données. On y trouve des fonctions SQL qui permettent par exemple de faire des requêtes pour récupérer, ajouter ou filtrer des données. Le dossier **controllers** contient la logique métier. C'est ici que les données reçues des utilisateurs sont traitées avant ou après un accès à la base via **models/**. Les fonctions dans les contrôleurs reçoivent les requêtes (**req**) et les réponses (**res**); cette partie fait l'interface entre les fichiers qui gèrent les interactions avec la base et l'affichage à l'utilisateur.

Le dossier **routes** contient les différentes routes de l'API. Chaque route correspond à une URL et une méthode HTTP (comme **GET** ou **POST**). Dans notre projet, les routes sont gérées à l'aide du framework Express.js, une bibliothèque pour Node.js. Express permet de définir simplement des routes qui appellent les fonctions correspondantes dans les contrôleurs. Par exemple, la route **/autocomplete** appelle la fonction **autocomplete** définie dans le contrôleur des parfums.

Cette organisation respecte le principe de séparation des responsabilités ("Separation of Concerns"). Le code est ainsi plus propre, modulaire et facile à tester. Si une fonctionnalité change, on peut modifier uniquement le fichier concerné sans toucher aux autres parties.

1.6 Gestion de l'état utilisateur

Afin de maintenir la session d'un utilisateur connecté, nous avons mis en place une variable globale persistante qui représente l'état de connexion. Cette variable contient un identifiant unique, stocké dans le **localStorage** du navigateur. Elle reste disponible tant que l'utilisateur ne se déconnecte pas.

Ce mécanisme nous permet d'identifier l'utilisateur à chaque interaction avec le backend, sans devoir redemander une authentification. La variable est lue dès le chargement initial de l'application via le hook **useEffect**, ce qui permet de reconnaître dynamiquement l'utilisateur tant qu'il ne se déconnecte pas.

Lorsque l'utilisateur choisit de se déconnecter, cette variable est simplement supprimée du **localStorage**, et l'état global de l'application est mis à jour en conséquence, le ramenant à la page de connexion.

Ce système léger et efficace nous a permis de gérer l'authentification de manière simple

et robuste.

1.7 Nettoyage et structuration de la base de données

L'une des premières étapes du projet a été la préparation de la base de données, qui s'est avérée essentielle pour la suite du traitement mathématique. Nous avons reçu un fichier CSV brut contenant uniquement des colonnes textuelles : *nom du parfum*, *notes de tête*, *notes de cœur*, *notes de fond*, *genre (homme/femme)* et *URL*. Ce format initial, bien que lisible, n'était pas exploitable directement pour les opérations matricielles nécessaires à la modélisation des préférences.

1.7.1 Indexation des entités textuelles

Pour permettre un traitement efficace, nous avons procédé à une transformation complète des données textuelles en données numériques :

- Chaque parfum a été identifié par un **index unique**, remplaçant son nom. Cela permet une représentation plus compacte et facilite les jointures ou associations avec d'autres tables.
- De la même manière, tous les composants olfactifs (tête, cœur, fond) ont été **indexés** numériquement. Par exemple, le composant "vanille" a été remplacé par un entier comme 42".
- Les informations de genre (homme/femme) ont également été codées sous forme binaire ou catégorielle.

Ce passage à une représentation numérique uniforme était nécessaire pour utiliser des matrices dans les algorithmes d'élicitation, d'optimisation ou de modélisation géométrique. En effet, il est beaucoup plus simple (et performant) de manipuler des entiers que des chaînes de caractères dans des structures comme les matrices A , les vecteurs x , ou pour des opérations comme les calculs de similarité ou d'agrégation.

1.7.2 Préparation de la base de données

Nous avons ensuite restructuré les données en quatre tableaux principaux, selon une logique de base de données relationnelle. Cela comprend une table des parfums contenant l'index du parfum, le genre et les liens vers les autres composants (tête, cœur, fond). Nous avons également une table des notes olfactives, où chaque ligne représente une note avec son index, son type (tête/cœur/fond) et son nom original. Une table de liaison parfum-notes établit le lien entre les parfums et leurs composants, permettant de représenter un parfum comme un ensemble de notes. Enfin, nous disposons aussi d'une table des métadonnées, qui contient des informations supplémentaires telles que l'URL ou d'autres attributs non essentiels pour la modélisation.

Grâce à cette structuration, chaque parfum est représenté comme un vecteur binaire ou pondéré sur l'ensemble des composants, ce qui a permis une vectorisation efficace pour les calculs de distances et de préférences. Ce nettoyage initial, bien que fastidieux, a été un pilier fondamental de la réussite du projet. Il a permis de rendre les données exploitables dans un cadre mathématique rigoureux, tout en préparant le terrain pour l'interfaçage avec l'utilisateur.

Chapitre 2

Élicitation des préférences et modélisation IA

2.1 Modélisation géométrique des préférences

Chaque comparaison exprimée par l'utilisateur (préférence de x_i sur x_j) est modélisée par une inégalité :

$$x_i < x_j \Rightarrow x_i - x_j < 0 \Rightarrow w^T x < 0$$

où a est un vecteur ayant $+1$ à la position i , -1 à la position j , et 0 ailleurs. L'ensemble de ces contraintes est regroupé dans une matrice $W \in \mathbb{R}^{m \times n}$, menant à un système :

$$Wx \leq 0$$

où W définit un polyèdre convexe P contenant tous les vecteurs $x = (x_1, x_2, \dots, x_n)$ compatibles avec les préférences exprimées :

$$\mathcal{P} = \{w \in \mathbb{R}^d \mid w \leq 0\}$$

Exemple de matrice de contraintes

Supposons que l'utilisateur ait exprimé les préférences suivantes :

- $x_1 < x_2$ (le parfum 2 est préféré au parfum 1)
- $x_3 < x_2$
- $x_4 < x_1$

Cela se traduit par une matrice $W \in \mathbb{R}^{3 \times 4}$ de la forme :

$$W = \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ -1 & 0 & 0 & 1 \end{bmatrix}$$

Chaque ligne représente une contrainte :

$$x_1 - x_2 < 0$$

$$x_3 - x_2 < 0$$

$$x_4 - x_1 < 0$$

Pourquoi passer d'un polytope à un polyèdre fermé ?

Dans cette partie, nous allons expliquer ce qu'est un **polytope** et un **polyèdre**, et pourquoi il est nécessaire de passer de l'un à l'autre dans notre contexte.

Comme on peut le voir sur la figure 2.1, toutes nos contraintes passent par le centre. Si l'on représente ce polytope en deux dimensions à partir de ces contraintes, on observe que la région définie est ouverte et tend vers l'infini.

Ce polytope est non borné vers le haut et la droite.

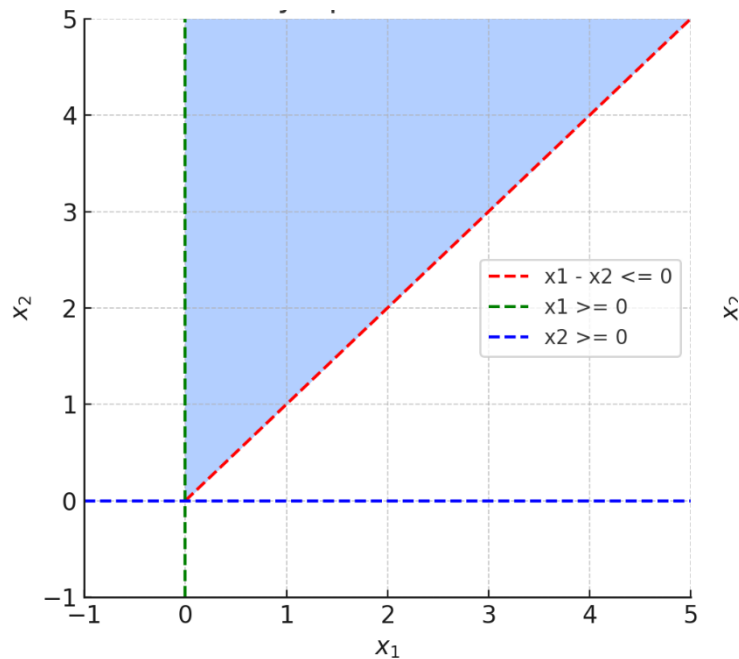


FIGURE 2.1 – Polytope

Or, dans la suite de notre algorithme, nous avons besoin de **trouver le centre du polytope**. Si celui-ci n'est pas fermé, l'algorithme (comme une descente de gradient) peut diverger vers l'infini, car il n'existe pas de centre bien défini dans un ensemble non borné.

Il est donc nécessaire de **fermer le polytope**, c'est-à-dire de l'encadrer dans une région finie. Une méthode couramment utilisée consiste à ajouter un carré (ou plus généralement un pavé) qui vient **encercler le polytope** pour le rendre borné, comme illustré dans la figure 2.2.

Nous allons maintenant réaliser la démonstration qui explique pourquoi on peut passer d'un polytope à un polyèdre.

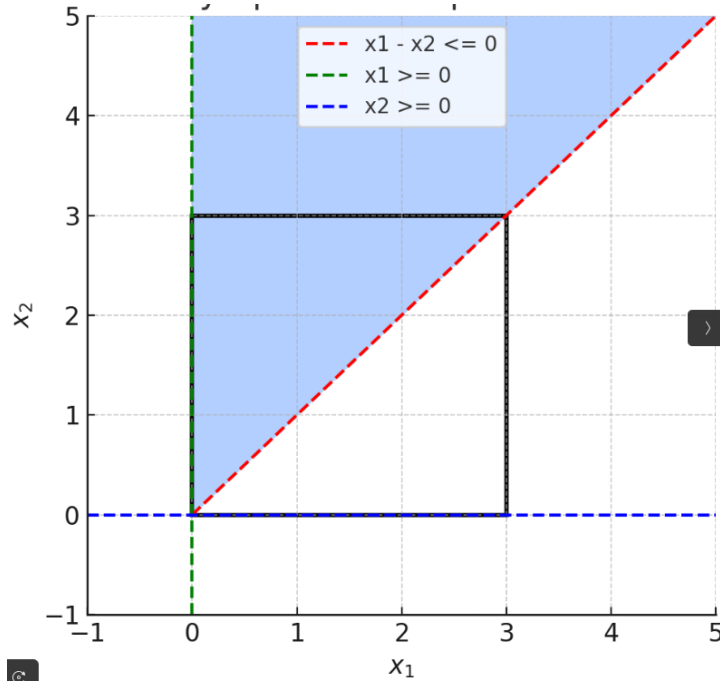


FIGURE 2.2 – Polyèdre fermé

2.2 Optimisation par fonction barrière logarithmique

Pour cela, nous avons utilisé la fonction barrière. En effet, plus on est proche des bords du polyèdre, plus la valeur de la fonction barrière est élevée ; et plus on est proche du centre, plus cette valeur est faible. À partir de cela, on peut essayer de trouver le centre du polyèdre grâce à une descente de gradient sur la fonction barrière. Celle-ci s'écrit :

$$\phi(x) = - \sum_{i=1}^m \log(-w_i^T x) \quad (2.1)$$

Gradient :

$$\nabla \phi(x) = \sum_{i=1}^m \frac{w_i}{w_i^T x}$$

Hessienne :

$$\nabla^2 \phi(x) = \sum_{i=1}^m \frac{w_i w_i^T}{(w_i^T x)^2}$$

Deux approches ont été mises en œuvre pour minimiser la fonction barrière et ainsi trouver le centre analytique du polyèdre : la descente de gradient, implémentée à l'aide de la bibliothèque `PyTorch`, et la méthode de Newton. Cette méthode consiste à suivre la direction opposée au gradient de la fonction barrière afin de réduire progressivement sa valeur. À chaque itération, la solution est mise à jour selon la règle :

$$x_{k+1} = x_k - \alpha \nabla \phi(x_k)$$

où α est un pas d'apprentissage fixé. Cette méthode est simple à implémenter et permet de

converger vers une solution en effectuant un grand nombre de petites mises à jour. Cette seconde méthode utilise non seulement le gradient, mais aussi la hessienne de la fonction barrière pour déterminer une direction de descente plus efficace. À chaque itération, on résout un système linéaire pour calculer l'incrément :

$$\nabla^2 \phi(x_k) \cdot \Delta x = \nabla \phi(x_k)$$

puis on met à jour la solution :

$$x_{k+1} = x_k - t\Delta x$$

où t est un coefficient déterminé par une recherche linéaire (line search) afin de garantir que le nouveau point reste dans le domaine de définition de la fonction barrière, c'est-à-dire à l'intérieur du polyèdre. Cette méthode offre une convergence quadratique rapide à proximité du minimum, mais son coût de calcul est plus élevé à chaque itération, en raison du calcul de la hessienne et de la résolution du système linéaire associé. La descente de gradient implémentée avec `PyTorch` est particulièrement adaptée aux problèmes de grande dimension, car elle ne requiert que le calcul du gradient. En revanche, la méthode de Newton présente une convergence plus rapide, mais au prix d'une complexité plus importante à chaque itération.

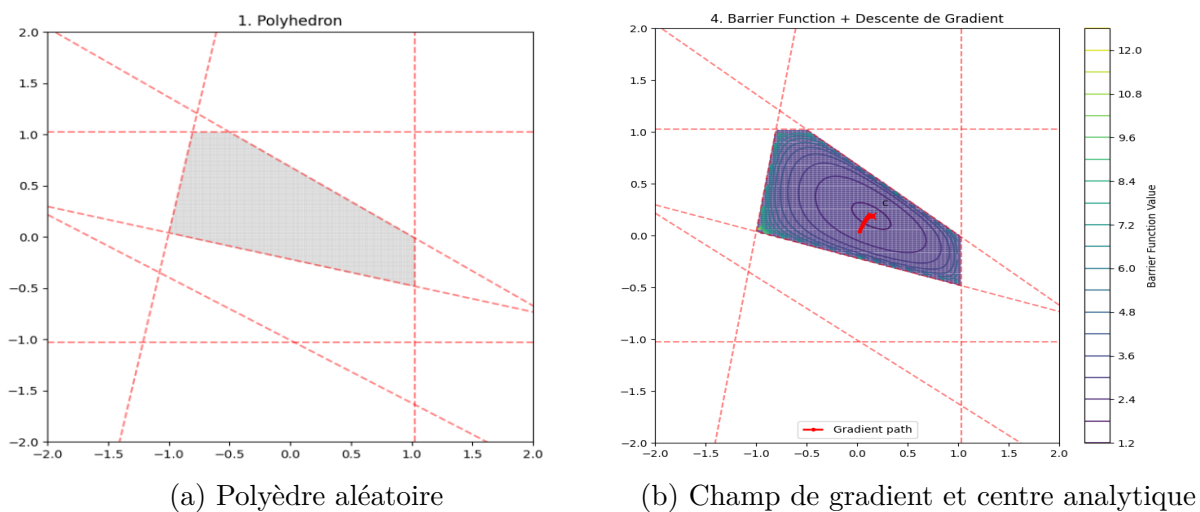


FIGURE 2.3 – Calcul du centre du polyèdre

La figure 2.3b montre le champ de gradient d'un polyèdre aléatoire (figure 2.3a), et le centre, qui minimise la fonction barrière 2.1.

2.3 Estimation locale : Ellipse de confiance

On souhaite tracer la plus petite ellipse qui entoure notre polyèdre. Pour cela, on va tracer l'ellipse ayant pour centre x^* . x^* étant le centre de notre polyèdre trouvé grâce à

nos contraintes. Pour calculer l'ellipse, on aura besoin de calculer l'inverse de la hessienne de la fonction barrière, ce qui nous permettra de tracer l'ellipse.

L'ellipse est définie par :

$$\{x \in \mathbb{R}^n \mid (x - x^*)^T H(x - x^*) \leq \delta^2\}$$

Elle capture la région d'incertitude autour des préférences de l'utilisateur.

Cette ellipse va être utile par la suite. En effet, nous allons maintenant implémenter un algorithme de meilleure question, c'est-à-dire proposer les deux meilleurs parfums à l'utilisateur pour réduire au maximum le polyèdre.

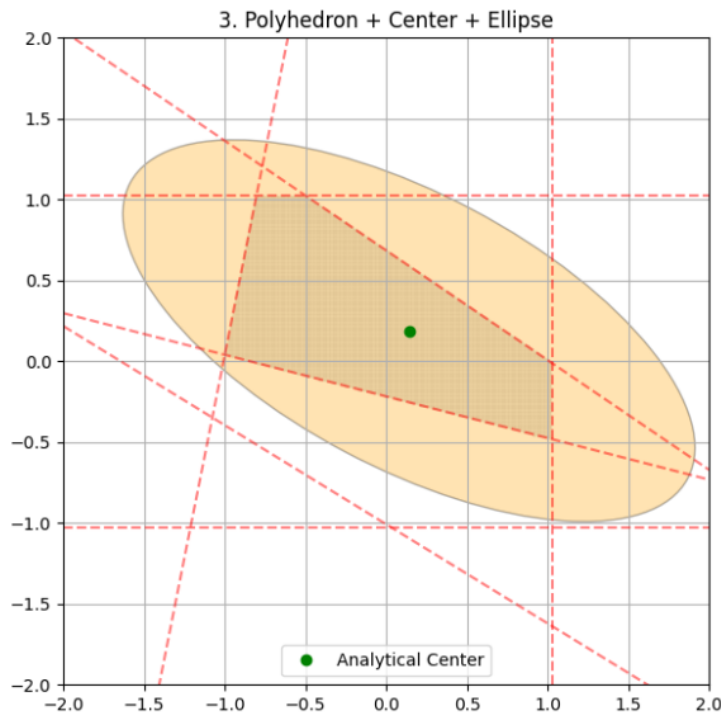


FIGURE 2.4 – Ellipse de confiance

2.4 Détermination de la meilleure question à poser

L'objectif est de trouver une question qui sépare le polyèdre admissible en deux parties à peu près égales, afin d'avoir un polyèdre plus petit quelle que soit la réponse de l'utilisateur. Le cas optimal est la question qui divise le mieux le polyèdre, parmi toutes les questions possibles.

Sur la figure 2.5a, la nouvelle question (en bleu) est mauvaise : la contrainte divise le polyèdre en deux parties très inégales. Si l'utilisateur se place au-dessus de la ligne bleue, alors le nouveau polyèdre n'a que très peu diminué en taille. Sur la figure 2.5b, la nouvelle question est bonne : elle est très proche du centre, et orthogonale au grand axe de l'ellipse.

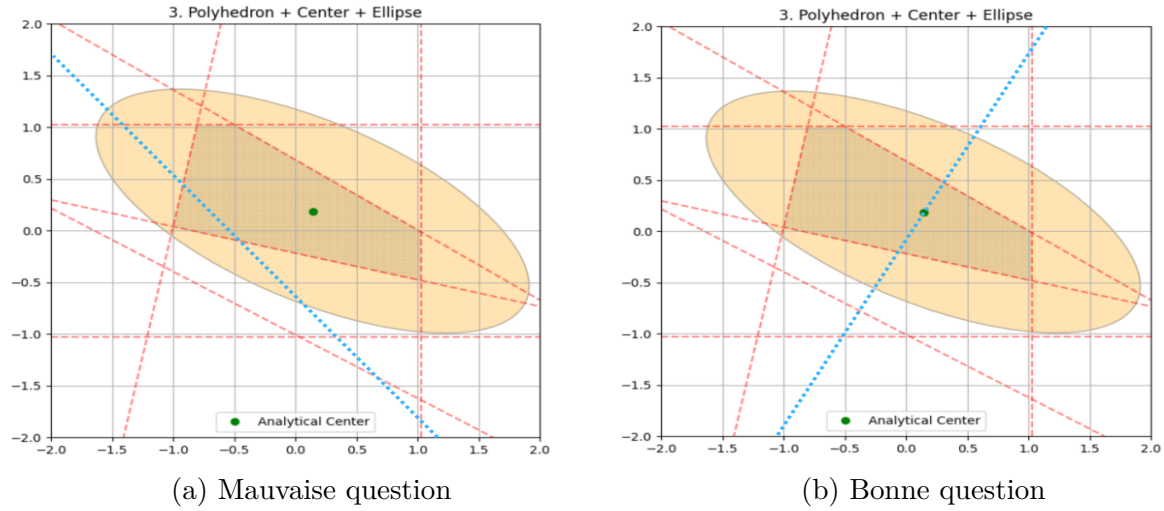


FIGURE 2.5 – Meilleure question à poser

2.4.1 Méthode de recherche exhaustive

La première méthode que nous avons implémenté est la recherche exhaustive. Nous avons construit une fonction de "fitness", qui attribue un score à chaque comparaison possible, et sélectionne la meilleure.

On construit d'abord tous les vecteurs de comparaison, c'est-à-dire tous les couples $x - y$, avec x et y des parfums. Ensuite, pour chaque vecteur, on calcule la distance au centre du polyèdre : plus il est proche du centre, plus il a de chances de bien couper le polyèdre. On calcule également le produit scalaire entre le vecteur et le grand axe de l'ellipse : si le produit scalaire est proche de 0, alors on est presque orthogonal au grand axe, et on sépare le polyèdre en deux parties presque égales. À l'inverse, si on est proche de 1 ou -1 , alors on est quasiment colinéaires avec le grand axe, et la coupe du polyèdre est bonne.

Le score renvoyé pour chaque vecteur est donc s_i (avec x^* le centre du polyèdre, v_i le vecteur de comparaison considéré, et u le grand axe de l'ellipse) :

$$d_i = \frac{\sum_j x_j^* \cdot v_j}{\|v\|}$$

$$p_i = \left| \frac{1}{2} - \langle v_i | u \rangle \right|$$

$$s_i = d_i + p_i$$

On calcule ce score pour chaque vecteur, et on garde celui qui a le plus faible : il s'agit de la meilleure question à poser. Cependant, cette méthode est très coûteuse en temps, à cause du nombre de vecteurs comparaisons : $\binom{1984}{2}$.

2.4.2 Méthode du critère D-optimality

Le but est de **réduire le volume du polyèdre admissible**, ce qui revient à vouloir réduire l'aire au maximum.

Le **critère D-optimality** (ou D-criterion) est une approche issue de la théorie du design expérimental optimal. Il consiste à choisir les nouvelles observations (ou contraintes) qui minimisent l'incertitude globale estimée autour de la solution. Mathématiquement, on s'appuie sur la hessienne $H^{-1} = \nabla^2 \phi(x^*)$ de la fonction barrière, qui définit localement une ellipse de confiance autour de la solution actuelle.

Le volume de cette ellipse est proportionnel à :

$$\text{Vol(ellipse)} \propto \frac{1}{\sqrt{\det(H)}}$$

Minimiser le volume revient donc à **maximiser** $\det(H)$. C'est cela qu'on appelle le **D-criterion** : sélectionner les comparaisons qui vont **maximiser le déterminant de la matrice d'information (ici H)**. Étant donné le très grand nombre de paires possibles ($\binom{1984}{2}$), il n'est pas envisageable de toutes les tester. Ce calcul d'aire est un excellent moyen pour nous de trouver une question qui n'est pas forcément la meilleure, mais qui est satisfaisante.

En pratique, il s'agit de comparer l'aire approchée du polyèdre initial et celle du polyèdre séparé par la nouvelle contrainte. Si le ratio $\frac{\text{Nouvelle aire}}{\text{Aire initiale}}$ est proche de $\frac{1}{2}$, on a divisé le polyèdre en deux parties de taille équivalentes. La question permet donc d'avoir un espace des possibilités plus petit, peu importe la réponse de l'utilisateur. Par exemple, si l'on sait que l'aire de l'ellipse est de 100, on peut dire que l'on a trouvé une bonne question si l'aire après la réponse de l'utilisateur est de 60. On n'a pas la meilleure question, mais on a une question qui divise suffisamment le polyèdre. Cela nous permet de réduire le temps de calcul et d'afficher à l'utilisateur de nouvelles questions plus rapidement.

2.5 Critère d'arrêt basé sur le Min-Max Regret

Une fois que suffisamment de préférences ont été exprimées par l'utilisateur, il devient inutile (ou même contre-productif) de continuer à poser des questions. Il faut donc un critère formel pour décider quand arrêter l'interrogation. Nous avons choisi d'utiliser le **min-max regret**, un critère issu de la théorie de la décision. Le **regret** associé à une option x_i est défini comme la différence entre le score de la meilleure option possible compte tenu des contraintes connues, et le score de x_i dans le pire des cas compatibles avec ces contraintes. Formellement, pour un vecteur de préférences x appartenant au polyèdre admissible $P = \{x \in \mathbb{R}^n \mid Ax \leq 0\}$, le regret maximal pour une option i est

donné par :

$$\text{Regret}(i) = \max_{x \in P} \left(\max_j x_j - x_i \right)$$

L'objectif est alors de minimiser ce pire cas : on cherche l'option dont le **regret maximal** est le plus petit.

On définit :

$$\text{MMR} = \min_i \max_{x \in P} \left(\max_j x_j - x_i \right)$$

L'indice i^* qui minimise ce critère correspond à la **meilleure option à recommander** selon les informations disponibles.

Là aussi, nous avons décidé de fixer un min-max regret à atteindre pour proposer le parfum. Une fois que ce seuil est atteint, l'algorithme juge que l'on a posé assez de questions et propose à l'utilisateur le parfum.

Nous avons mis en place l'algorithme suivant pour chaque parfum i , on résout un problème d'optimisation :

$$\text{Regret}(i) = \max_{x \in P} \left(\max_j x_j - x_i \right)$$

On calcule ensuite le $\text{MMR} = \min_i \text{Regret}(i)$. Si ce MMR est inférieur à un seuil ε , on arrête les questions et on recommande le parfum i^* correspondant. Cette méthode garantit que nous ne recommandons un parfum que lorsque le risque de regret (lié à l'incertitude résiduelle) est suffisamment faible, ce qui améliore la **qualité de la recommandation finale** tout en **limitant le nombre de questions posées**.

Chapitre 3

Bilan du développement : réussites et difficultés

Ce chapitre présente un retour d'expérience sur le projet, en distinguant ce qui a bien fonctionné, ce qui a posé problème, et les leçons tirées au cours du développement. Notre équipe s'est organisée en deux binômes au départ, avec une répartition entre la partie interface web et la partie modélisation mathématique.

3.1 Organisation initiale et réajustements

L'interface web a été relativement rapide à implémenter, avec d'outils bien maîtrisés. À l'inverse, la partie modélisation mathématique a accumulé des retards importants, en raison de la complexité mathématique et du manque d'expérience sur les outils utilisés. Pour cette raison, nous avons restructuré la répartition des tâches et nous nous sommes principalement concentrés sur les parties importantes.

3.2 Difficultés rencontrées

3.2.1 Exploration de plusieurs approches non concluantes

L'approche du critère D-optimality était prometteuse, mais nous n'avons pas réussi à la faire fonctionner : le déterminant de la hessienne est soit nul, soit trop grand pour être représenté, selon la méthode utilisée pour borner le polyèdre.

La descente de gradient implémentée en utilisant la bibliothèque `PyTorch` s'est révélée trop lente à converger, notamment à cause de la taille du problème (on minimise une fonction dans un espace de dimension 328). Cela a entraîné une perte de temps considérable et a ralenti la mise en place des étapes suivantes.

3.2.2 Lenteur générale de la modélisation au départ

La modélisation initiale du polyèdre, du centre et de l'ellipse a pris beaucoup plus de temps que prévu. La complexité mathématique, combinée à un manque d'expérience, a abouti à des implémentations incomplètes ou incorrectes. Ce n'est qu'après plusieurs essais que toute la modélisation a commencé à fonctionner correctement.

3.3 Réussites techniques et méthodologiques

3.3.1 Méthode de Newton pour la fonction barrière

Le remplacement de la descente de gradient par la méthode de Newton a constitué un tournant majeur dans le projet. Cette méthode, bien que plus complexe à mettre en œuvre, s'est montrée beaucoup plus efficace et plus performante. Elle a permis de calculer le centre du polyèdre de manière fiable, débloquent ainsi toute la suite du pipeline (ellipse, meilleure question, etc.).

3.3.2 Recherche exhaustive des meilleures questions

Nous avons implémentés beaucoup de méthodes différentes pour le choix des meilleures questions, qui ont été assez longue à réalisées. Bien que coûteuse en calcul, ces méthodes ont offert les meilleurs résultats en termes de qualité de recommandation. Ce choix a été l'un des grands succès du projet.

3.3.3 Implémentation du min-max regret

Le critère de **Min-Max Regret** est utilisé pour identifier, à tout moment de l'élicitation, l'alternative (parfum) ayant les meilleures garanties quelles que soient les préférences exactes de l'utilisateur.

Formellement, pour chaque alternative x , on calcule :

$$\max_{w \in \mathcal{W}} \left(\max_{y \in \mathcal{X}} f_w(y) - f_w(x) \right)$$

où $\mathcal{W}$ est l'ensemble des poids compatibles avec les comparaisons utilisateur, et $f_w(x) = w^\top x$.

L'alternative avec le plus petit regret maximal est sélectionnée.

Lors de la première implémentation, l'algorithme code estimait le regret en itérant sur des vecteurs extraits directement de la matrice de contraintes W . Or, ces vecteurs ne sont pas garantis d'être dans le polyèdre $\mathcal{W}$. Résultat : les regrets calculés ne reflétaient pas la vraie incertitude restante, et pouvaient même croître au fil des questions. La version fidèle du calcul consiste à résoudre un programme linéaire pour chaque comparaison potentielle,

mais cela s'est révélé trop coûteux en temps de calcul, notamment à chaque itération de l'interaction.

Nous avons donc choisi comme solution une méthode approchée : on échantillonne aléatoirement des vecteurs $x \in \mathcal{W}$, et on ne conserve que ceux qui respectent $W \cdot w \geq \varepsilon$ (W est la matrice des contraintes, w est le nouveau vecteur). On calcule ensuite le regret maximal de ce nuage de poids.

Cette méthode permet un temps de calcul très rapide tout en respectant les contraintes de préférence.

Le graphique ci-dessous montre l'évolution du regret au cours des itérations selon deux stratégies :



FIGURE 3.1 – Évolution du regret

On voit que la stratégie par meilleure question (bleue et orange) réduit rapidement le regret, souvent en quelques itérations. La stratégie aléatoire converge aussi, mais de manière bien plus lente, voire stagnante. Cela confirme que choisir intelligemment les questions à poser améliore significativement l'efficacité de l'élicitation.

3.3.4 Alternative au regret

Pour avoir une autre vision sur l'efficacité de nos méthodes, nous avons également implémenté une mesure de la proximité des parfums proposés dans la nouvelle question, basée sur la proportion d'ingrédients en commun dans chacune des notes (tête, cœur, fond) du parfum.

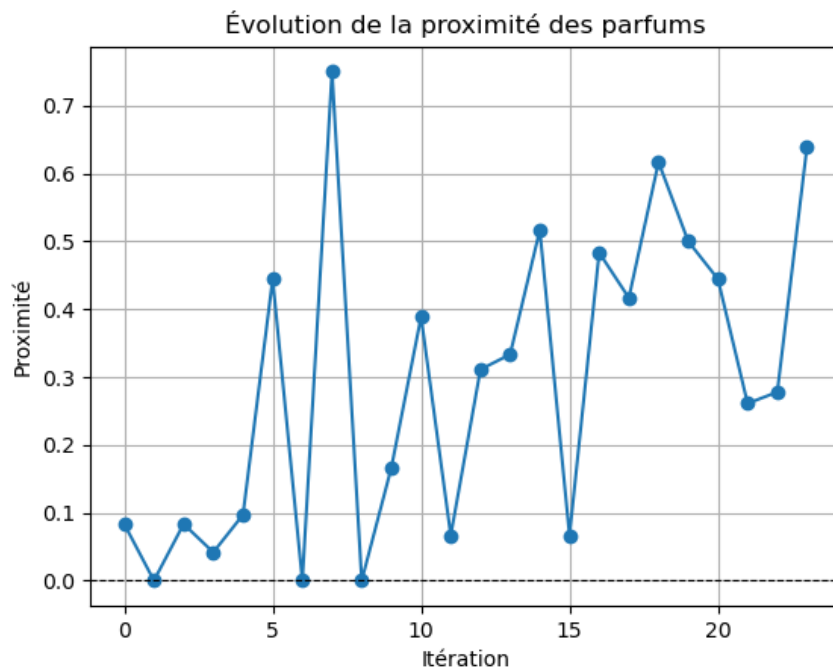


FIGURE 3.2 – Proximité des parfums

Malgré de fortes variations dues au fait que l'algorithme peut choisir deux parfums très proches dès les premières itérations, on constate une augmentation de la proportion d'ingrédients en commun entre les deux parfums proposés. Cela montre que l'algorithme affine l'espace des possibles en comparant deux parfums de plus en plus proches au cours des itérations.

3.4 Leçons retenues

Ce projet a mis en lumière l'importance d'une répartition efficace des tâches, mais aussi la nécessité de s'adapter lorsque certaines parties plus complexes sont laissées sans solutions. L'intervention croisée entre les sous-groupes a été très importante. De plus, il a été essentiel d'accepter d'abandonner ou de modifier certaines idées prometteuses sur le papier (comme la D-optimality ou la descente de gradient) pour revenir à des méthodes plus simples mais plus robustes.

Ce retour d'expérience met en évidence que les projets techniques mêlant développement et mathématiques nécessitent une forte collaboration, des tests rapides, et la capacité à remettre en question certaines décisions.

Conclusion

Ce projet a été l’occasion de concevoir un système complet d’aide à la décision, combinant modélisation mathématique avancée et développement web, autour de la recommandation personnalisée de parfums. Nous avons mis en œuvre une approche d’éllicitation incrémentale des préférences, permettant de capter et modéliser les goûts olfactifs d’un utilisateur à travers des questions et des évaluations de parfums. Grâce à une base de données structurée, et à des outils d’optimisation tels que la fonction barrière, la méthode de Newton ou encore le critère du min-max regret, nous avons développé un algorithme capable de fournir des recommandations pertinentes. Pour accompagner cet algorithme, nous avons conçu une interface web fluide et intuitive, en utilisant React, Flask et Vite. Ce choix technologique nous a permis d’offrir une expérience utilisateur optimale, accessible et agréable. Au-delà du cas spécifique de la parfumerie, ce projet ouvre des perspectives plus larges : l’éllicitation incrémentale peut s’appliquer à de nombreux domaines où les préférences sont subjectives, complexes ou difficilement formulables.

Annexes

Lien du dépôt Github : [dépôt](#). Le code utilisé dans la section 2 se trouve dans le dossier `site_react/src/backend/`.